



UNIVERSIDAD NACIONAL DEL ALTIPLANO

CURSO:

Programación Numérica

DOCENTE:

Ing. Torres Cruz Fredi

ESTUDIANTE:

Milena Kely Churata Mamani

TRABAJO_12

Interpolación: Métodos y Fórmulas de Alta Precisión

Puno, Perú

21 de diciembre de 2025

Resumen

Este documento presenta una guía completa sobre métodos de interpolación numérica, incluyendo interpolación lineal, polinomios de Lagrange, diferencias divididas de Newton, interpolación cuadrática y splines cúbicos. Para cada método, se proporcionan tres ejemplos detallados con soluciones matemáticas y código R implementado. El objetivo es ofrecer una comprensión práctica de las técnicas de interpolación para aplicaciones en ciencia de datos, ingeniería y análisis numérico.

Índice

1. Introducción	3
1.1. Aplicaciones Prácticas	3
2. Interpolación Lineal	3
2.1. Fórmula General	3
2.2. Ejemplo 1: Temperatura Diaria	3
2.3. Ejemplo 2: Crecimiento de Inversión	3
2.4. Ejemplo 3: Velocidad de Caída Libre	3
2.5. Código R - Interpolación Lineal	4
3. Interpolación de Lagrange	5
3.1. Fórmula General	5
3.2. Ejemplo 1: Tres Puntos Simétricos	5
3.3. Ejemplo 2: Cuatro Puntos	5
3.4. Ejemplo 3: Puntos Equiespaciados	5
3.5. Código R - Interpolación de Lagrange	5
4. Diferencias Divididas de Newton	7
4.1. Fórmula General	7
4.2. Ejemplo 1: Tres Puntos	7
4.3. Ejemplo 2: Cuatro Puntos	7
4.4. Ejemplo 3: Puntos No Equiespaciados	7
4.5. Código R - Diferencias Divididas de Newton	8
5. Interpolación Cuadrática	10
5.1. Fórmula General	10
5.2. Ejemplo 1: Trayectoria Parabólica	10
5.3. Ejemplo 2: Ventas Trimestrales	10
5.4. Ejemplo 3: Mediciones de Temperatura	10
5.5. Código R - Interpolación Cuadrática	10
6. Splines Cúbicos	12
6.1. Fórmula General	12
6.2. Ejemplo 1: Tres Puntos Simples	12
6.3. Ejemplo 2: Cuatro Puntos de Control	12
6.4. Ejemplo 3: Datos Experimentales	12
6.5. Código R - Splines Cúbicos	12
7. Conclusión	14
7.1. Resumen Comparativo de Métodos	14
7.2. Recomendaciones de Uso	14
7.3. Aplicaciones Prácticas	14
7.4. Perspectivas Futuras	14
7.5. Código Final Integrado	15

1. Introducción

La interpolación es una técnica fundamental en análisis numérico que permite estimar valores desconocidos dentro de un rango de datos conocidos. A diferencia de la extrapolación (que estima valores fuera del rango conocido), la interpolación proporciona estimaciones más confiables al mantenerse dentro del dominio de los datos observados.

1.1. Aplicaciones Prácticas

- Predicción de temperaturas entre mediciones horarias
- Estimación de ventas entre puntos de datos mensuales
- Reconstrucción de señales en procesamiento digital
- Generación de curvas suaves en diseño gráfico
- Análisis de trayectorias en física y robótica

2. Interpolación Lineal

2.1. Fórmula General

Para dos puntos (x_0, y_0) y (x_1, y_1) , el valor interpolado en x es:

$$y = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0)$$

2.2. Ejemplo 1: Temperatura Diaria

Datos: Temperaturas registradas: 8:00 AM (20°C), 12:00 PM (28°C)

Problema: Estimar la temperatura a las 10:00 AM

Solución:

$$x_0 = 8, y_0 = 20; \quad x_1 = 12, y_1 = 28; \quad x = 10$$
$$y = 20 + \frac{28 - 20}{12 - 8}(10 - 8) = 20 + \frac{8}{4} \times 2 = 20 + 4 = 24^\circ\text{C}$$

2.3. Ejemplo 2: Crecimiento de Inversión

Datos: Inversión inicial: \$1000 (mes 1), \$1500 (mes 6)

Problema: Estimar valor en mes 3

Solución:

$$y = 1000 + \frac{1500 - 1000}{6 - 1}(3 - 1) = 1000 + \frac{500}{5} \times 2 = 1000 + 200 = \$1200$$

2.4. Ejemplo 3: Velocidad de Caída Libre

Datos: Velocidad: 0 m/s (t=0s), 19.6 m/s (t=2s)

Problema: Estimar velocidad en t=1.5s

Solución:

$$y = 0 + \frac{19.6 - 0}{2 - 0}(1.5 - 0) = 9.8 \times 1.5 = 14.7 \text{ m/s}$$

2.5. Código R - Interpolación Lineal

```
1 # Funci n para interpolaci n lineal
2 interpolacion_lineal <- function(x0, y0, x1, y1, x) {
3   y <- y0 + ((y1 - y0) / (x1 - x0)) * (x - x0)
4   return(y)
5 }
6
7 # Ejemplo 1: Temperatura
8 temp <- interpolacion_lineal(8, 20, 12, 28, 10)
9 cat("Temperatura a las 10:00 AM:", temp, " C \n")
10
11 # Ejemplo 2: Inversi n
12 inversion <- interpolacion_lineal(1, 1000, 6, 1500, 3)
13 cat("Valor de inversi n en mes 3: $", inversion, "\n")
14
15 # Ejemplo 3: Velocidad
16 velocidad <- interpolacion_lineal(0, 0, 2, 19.6, 1.5)
17 cat("Velocidad en t=1.5s:", velocidad, "m/s\n")
18
19 # Funci n generalizada para m ltiples puntos
20 interpolar_vector <- function(x_vals, y_vals, x_nuevo) {
21   n <- length(x_vals)
22   for (i in 1:(n-1)) {
23     if (x_nuevo >= x_vals[i] && x_nuevo <= x_vals[i+1]) {
24       return(interpolacion_lineal(x_vals[i], y_vals[i],
25                                   x_vals[i+1], y_vals[i
26                                     +1],
27                                   x_nuevo))
28     }
29   }
30   return(NA)
31 }
32
33 # Prueba con datos m ltiples
34 x_datos <- c(8, 12, 16)
35 y_datos <- c(20, 28, 25)
36 cat("Temperatura a las 10:00 AM:", interpolar_vector(x_datos, y
37 _datos, 10), " C \n")
```

3. Interpolación de Lagrange

3.1. Fórmula General

Para $n + 1$ puntos (x_i, y_i) :

$$P(x) = \sum_{i=0}^n y_i L_i(x), \quad \text{donde } L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

3.2. Ejemplo 1: Tres Puntos Simétricos

Datos: (0, 1), (2, 5), (4, 3)

Problema: Calcular $P(1)$

Solución:

$$L_0(1) = \frac{(1-2)(1-4)}{(0-2)(0-4)} = \frac{3}{8}$$

$$L_1(1) = \frac{(1-0)(1-4)}{(2-0)(2-4)} = \frac{3}{4}$$

$$L_2(1) = \frac{(1-0)(1-2)}{(4-0)(4-2)} = -\frac{1}{8}$$

$$P(1) = 1 \times \frac{3}{8} + 5 \times \frac{3}{4} + 3 \times \left(-\frac{1}{8}\right) = 4$$

3.3. Ejemplo 2: Cuatro Puntos

Datos: (-1, 2), (0, 1), (1, 0), (2, 5)

Problema: Calcular $P(0,5)$

Solución:

$$L_0(0,5) = \frac{(0,5-0)(0,5-1)(0,5-2)}{(-1-0)(-1-1)(-1-2)} = -0,0625 L_1(0,5) = \frac{(0,5+1)(0,5-1)(0,5-2)}{(0+1)(0-1)(0-2)} = 0,5625 L_2(0,5) = \frac{(0,5-1)(0,5-2)(0,5-3)}{(-1-1)(-1-2)(-1-3)} = -0,1875 L_3(0,5) = \frac{(0,5-0)(0,5-1)(0,5-2)}{(2-0)(2-1)(2-2)} = 0$$

3.4. Ejemplo 3: Puntos Equiespaciados

Datos: (1, 1), (2, 4), (3, 9), (4, 16)

Problema: Calcular $P(2,5)$

Solución:

$$L_0(2,5) = \frac{(2,5-2)(2,5-3)(2,5-4)}{(1-2)(1-3)(1-4)} = -0,125 L_1(2,5) = \frac{(2,5-1)(2,5-3)(2,5-4)}{(2-1)(2-3)(2-4)} = 0,75 L_2(2,5) = \frac{(2,5-1)(2,5-2)(2,5-4)}{(3-1)(3-2)(3-4)} = -0,375 L_3(2,5) = \frac{(2,5-1)(2,5-2)(2,5-3)}{(4-1)(4-2)(4-3)} = 0,125$$

3.5. Código R - Interpolación de Lagrange

```
1 # Funci n para calcular polinomio base de Lagrange
2 lagrange_base <- function(x, i, x_vals) {
3   n <- length(x_vals)
4   producto <- 1
5   for (j in 1:n) {
6     if (j != i) {
7       producto <- producto * (x - x_vals[j]) / (x_vals[i]
8       - x_vals[j])
9     }
10  }
```

```

9     }
10     return(producto)
11 }
12
13 # Funci n principal de interpolaci n de Lagrange
14 interpolacion_lagrange <- function(x_vals, y_vals, x) {
15     n <- length(x_vals)
16     resultado <- 0
17     for (i in 1:n) {
18         L_i <- lagrange_base(x, i, x_vals)
19         resultado <- resultado + y_vals[i] * L_i
20     }
21     return(resultado)
22 }
23
24 # Ejemplo 1: Tres puntos
25 x1 <- c(0, 2, 4)
26 y1 <- c(1, 5, 3)
27 cat("Ejemplo 1 - P(1):", interpolacion_lagrange(x1, y1, 1), "\n")
28
29 # Ejemplo 2: Cuatro puntos
30 x2 <- c(-1, 0, 1, 2)
31 y2 <- c(2, 1, 0, 5)
32 cat("Ejemplo 2 - P(0.5):", interpolacion_lagrange(x2, y2, 0.5),
33     "\n")
34
35 # Ejemplo 3: Puntos cuadr ticos
36 x3 <- c(1, 2, 3, 4)
37 y3 <- c(1, 4, 9, 16)
38 cat("Ejemplo 3 - P(2.5):", interpolacion_lagrange(x3, y3, 2.5),
39     "\n")
40
41 # Visualizaci n de polinomio
42 x_seq <- seq(min(x3), max(x3), length.out = 100)
43 y_seq <- sapply(x_seq, function(x) interpolacion_lagrange(x3,
44     y3, x))
45
46 plot(x3, y3, pch = 19, col = "red", cex = 1.5,
47     main = "Interpolaci n de Lagrange",
48     xlab = "x", ylab = "f(x)")
49 lines(x_seq, y_seq, col = "blue", lwd = 2)
50 legend("topleft", legend = c("Datos", "Polinomio"),
51     col = c("red", "blue"), pch = c(19, NA), lty = c(NA, 1))

```

4. Diferencias Divididas de Newton

4.1. Fórmula General

Polinomio interpolante:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots$$

donde:

$$f[x_i] = y_i, \quad f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

4.2. Ejemplo 1: Tres Puntos

Datos: (1, 2), (2, 5), (4, 17)

Problema: Encontrar polinomio interpolante

Solución - Tabla de diferencias:

x	$f[]$	$f[,]$	$f[, ,]$
1	2	3	2
2	5	6	
4	17		

$$P(x) = 2 + 3(x - 1) + 2(x - 1)(x - 2) = 2x^2 - 3x + 3$$

4.3. Ejemplo 2: Cuatro Puntos

Datos: (0, 1), (1, 3), (2, 7), (4, 21)

Problema: Encontrar $P(1,5)$

Solución:

x	$f[]$	$f[,]$	$f[, ,]$	$f[, , ,]$
0	1	2	1	0
1	3	4	1	
2	7	7		
4	21			

$$P(x) = 1 + 2x + 1 \cdot x(x - 1) + 0 = x^2 + x + 1$$

$$P(1,5) = (1,5)^2 + 1,5 + 1 = 4,75$$

4.4. Ejemplo 3: Puntos No Equiespaciados

Datos: (1, 1), (3, 9), (6, 36), (10, 100)

Problema: Encontrar polinomio interpolante

Solución:

x	$f[]$	$f[,]$	$f[, ,]$
1	1	4	1
3	9	9	1
6	36	16	
10	100		

$$P(x) = 1 + 4(x - 1) + 1 \cdot (x - 1)(x - 3) = x^2$$

4.5. Código R - Diferencias Divididas de Newton

```
1  # Funci n para calcular tabla de diferencias divididas
2  tabla_diferencias <- function(x_vals, y_vals) {
3      n <- length(x_vals)
4      tabla <- matrix(0, n, n)
5      tabla[,1] <- y_vals
6
7      for (j in 2:n) {
8          for (i in 1:(n-j+1)) {
9              tabla[i,j] <- (tabla[i+1,j-1] - tabla[i,j-1]) / (x_
10                 vals[i+j-1] - x_vals[i])
11          }
12      }
13      return(tabla)
14  }
15
16  # Funci n para evaluar polinomio de Newton
17  interpolacion_newton <- function(x_vals, y_vals, x) {
18      n <- length(x_vals)
19      tabla <- tabla_diferencias(x_vals, y_vals)
20      resultado <- tabla[1,1]
21      producto <- 1
22
23      for (i in 2:n) {
24          producto <- producto * (x - x_vals[i-1])
25          resultado <- resultado + tabla[1,i] * producto
26      }
27      return(resultado)
28  }
29
30  # Ejemplo 1
31  x1 <- c(1, 2, 4)
32  y1 <- c(2, 5, 17)
33  cat("Ejemplo 1 - Tabla de diferencias:\n")
34  print(tabla_diferencias(x1, y1))
35  cat("P(3) =", interpolacion_newton(x1, y1, 3), "\n")
36
37  # Ejemplo 2
38  x2 <- c(0, 1, 2, 4)
39  y2 <- c(1, 3, 7, 21)
40  cat("\nEjemplo 2 - P(1.5) =", interpolacion_newton(x2, y2, 1.5)
41      , "\n")
42
43  # Ejemplo 3
44  x3 <- c(1, 3, 6, 10)
45  y3 <- c(1, 9, 36, 100)
46  cat("\nEjemplo 3 - P(5) =", interpolacion_newton(x3, y3, 5), "\n")
47
48  # Funci n para obtener coeficientes del polinomio
```



```
47 coeficientes_newton <- function(x_vals, y_vals) {  
48     n <- length(x_vals)  
49     tabla <- tabla_diferencias(x_vals, y_vals)  
50     coefs <- tabla[1,]  
51     return(coefs)  
52 }  
53  
54 cat("\nCoeficientes del polinomio (Ejemplo 1):\n")  
55 print(coeficientes_newton(x1, y1))
```

5. Interpolación Cuadrática

5.1. Fórmula General

Para tres puntos $(x_0, y_0), (x_1, y_1), (x_2, y_2)$, resolver:

$$\{ a x_0^2 + b x_0 + c = y_0, a x_1^2 + b x_1 + c = y_1, a x_2^2 + b x_2 + c = y_2$$

5.2. Ejemplo 1: Trayectoria Parabólica

Datos: $(0, 0), (1, 1), (2, 4)$

Problema: Encontrar ecuación de la parábola

Solución:

$$\{ c = 0a + b + c = 14a + 2b + c = 4$$

Resolviendo: $a = 1, b = 0, c = 0$, luego $P(x) = x^2$

5.3. Ejemplo 2: Ventas Trimestrales

Datos: Mes 1: 100 unidades, Mes 3: 400 unidades, Mes 5: 900 unidades

Problema: Estimar ventas en mes 4

Solución:

$$\{ a(1)^2 + b(1) + c = 100, a(3)^2 + b(3) + c = 400, a(5)^2 + b(5) + c = 900$$

Resolviendo: $a = 100, b = -100, c = 100$, luego $P(x) = 100x^2 - 100x + 100$

$P(4) = 100(16) - 100(4) + 100 = 1300$ unidades

5.4. Ejemplo 3: Mediciones de Temperatura

Datos: 8:00 (20°C), 10:00 (25°C), 12:00 (26°C)

Problema: Estimar temperatura a las 11:00

Solución: Usando x como horas desde 8:00:

$$\{ c = 204a + 2b + c = 2516a + 4b + c = 26$$

Resolviendo: $a = -0,5, b = 3,5, c = 20$

$P(3) = -0,5(9) + 3,5(3) + 20 = 26^\circ\text{C}$

5.5. Código R - Interpolación Cuadrática

```
1 # Funci n para interpolaci n cuadr tica
2 interpolacion_cuadratica <- function(x_vals, y_vals, x) {
3   # Sistema de ecuaciones: Ax = b
4   A <- matrix(c(x_vals[1]^2, x_vals[1], 1,
5                 x_vals[2]^2, x_vals[2], 1,
6                 x_vals[3]^2, x_vals[3], 1),
7               nrow = 3, byrow = TRUE)
8   b <- matrix(y_vals, ncol = 1)
9
10  # Resolver sistema
```

```

11     coeficientes <- solve(A, b)
12
13     # Evaluar en x
14     resultado <- coeficientes[1] * x^2 + coeficientes[2] * x +
15         coeficientes[3]
16     return(resultado)
17 }
18
19 # Ejemplo 1: Trayectoria parabólica
20 x1 <- c(0, 1, 2)
21 y1 <- c(0, 1, 4)
22 cat("Ejemplo 1 - P(1.5) =", interpolacion_cuadratica(x1, y1,
23     1.5), "\n")
24
25 # Ejemplo 2: Ventas
26 x2 <- c(1, 3, 5)
27 y2 <- c(100, 400, 900)
28 cat("Ejemplo 2 - Ventas en mes 4:", interpolacion_cuadratica(x2
29     , y2, 4), "unidades\n")
30
31 # Ejemplo 3: Temperatura
32 x3 <- c(0, 2, 4) # Horas desde 8:00
33 y3 <- c(20, 25, 26)
34 cat("Ejemplo 3 - Temperatura a las 11:00:",
35     interpolacion_cuadratica(x3, y3, 3), " C \n")
36
37 # Función para obtener coeficientes
38 coef_cuadratica <- function(x_vals, y_vals) {
39     A <- matrix(c(x_vals[1]^2, x_vals[1], 1,
40         x_vals[2]^2, x_vals[2], 1,
41         x_vals[3]^2, x_vals[3], 1),
42         nrow = 3, byrow = TRUE)
43     b <- matrix(y_vals, ncol = 1)
44     coeficientes <- solve(A, b)
45     return(as.vector(coeficientes))
46 }
47
48 # Visualización
49 x_seq <- seq(min(x2), max(x2), length.out = 100)
50 coefs <- coef_cuadratica(x2, y2)
51 y_seq <- coefs[1] * x_seq^2 + coefs[2] * x_seq + coefs[3]
52
53 plot(x2, y2, pch = 19, col = "red", cex = 1.5,
54     main = "Interpolación Cuadrática - Ventas",
55     xlab = "Mes", ylab = "Unidades vendidas")
56 lines(x_seq, y_seq, col = "blue", lwd = 2)
57 points(4, interpolacion_cuadratica(x2, y2, 4),
58     pch = 17, col = "green", cex = 2)
59 legend("topleft", legend = c("Datos", "Ajuste", "Predicción"),
60     col = c("red", "blue", "green"), pch = c(19, NA, 17),
61     lty = c(NA, 1, NA))

```

6. Splines Cúbicos

6.1. Fórmula General

Para cada intervalo $[x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

con condiciones de continuidad en valores y derivadas.

6.2. Ejemplo 1: Tres Puntos Simples

Datos: (0, 0), (1, 1), (2, 4)

Problema: Encontrar spline cúbico natural

Solución: Resolviendo el sistema (calculado con software): Los splines resultantes son funciones suaves que pasan por todos los puntos.

6.3. Ejemplo 2: Cuatro Puntos de Control

Datos: (0, 1), (1, 0), (2, 1), (3, 0)

Problema: Spline cúbico natural

Solución: El spline produce una curva ondulada suave que conecta los puntos.

6.4. Ejemplo 3: Datos Experimentales

Datos: (0, 0.5), (1, 1.2), (2, 0.8), (3, 1.5), (4, 2.0)

Problema: Ajuste suave para visualización

6.5. Código R - Splines Cúbicos

```
1 # Ejemplo 1: Spline c bico simple
2 x1 <- c(0, 1, 2)
3 y1 <- c(0, 1, 4)
4
5 # Usar funci n spline de R (spline c bico natural)
6 spline1 <- spline(x1, y1, n = 100)
7
8 plot(x1, y1, pch = 19, col = "red", cex = 1.5,
9      main = "Spline C bico Natural - Ejemplo 1",
10     xlab = "x", ylab = "y", ylim = c(-1, 5))
11 lines(spline1, col = "blue", lwd = 2)
12 grid()
13
14 # Ejemplo 2: Datos oscilantes
15 x2 <- c(0, 1, 2, 3)
16 y2 <- c(1, 0, 1, 0)
17
18 spline2 <- spline(x2, y2, n = 200)
19
20 plot(x2, y2, pch = 19, col = "red", cex = 1.5,
```

```

21     main = "Spline C bico - Datos Oscilantes",
22     xlab = "x", ylab = "y")
23 lines(spline2, col = "blue", lwd = 2)
24 grid()
25
26 # Ejemplo 3: Datos experimentales
27 x3 <- c(0, 1, 2, 3, 4)
28 y3 <- c(0.5, 1.2, 0.8, 1.5, 2.0)
29
30 spline3 <- spline(x3, y3, n = 200)
31
32 plot(x3, y3, pch = 19, col = "red", cex = 1.5,
33      main = "Spline C bico - Datos Experimentales",
34      xlab = "Tiempo", ylab = "Medici n")
35 lines(spline3, col = "blue", lwd = 2)
36 grid()
37
38 # Funci n para evaluar spline en un punto espec fico
39 evaluar_spline <- function(x_vals, y_vals, x_punto) {
40     spline_fit <- spline(x_vals, y_vals, n = 1000)
41     # Encontrar el valor m s cercano
42     idx <- which.min(abs(spline_fit$x - x_punto))
43     return(spline_fit$y[idx])
44 }
45
46 cat("Ejemplo 1 - Spline en x=1.5:", evaluar_spline(x1, y1, 1.5)
47     , "\n")
48 cat("Ejemplo 2 - Spline en x=1.5:", evaluar_spline(x2, y2, 1.5)
49     , "\n")
50 cat("Ejemplo 3 - Spline en x=2.5:", evaluar_spline(x3, y3, 2.5)
51     , "\n")
52
53 # Comparaci n de diferentes tipos de spline
54 par(mfrow = c(2, 2))
55
56 # Spline natural
57 plot(x3, y3, main = "Spline Natural", pch = 19)
58 lines(spline(x3, y3, method = "natural"), col = "red")
59
60 # Spline peri dico
61 plot(x3, y3, main = "Spline Peri dico", pch = 19)
62 lines(spline(x3, y3, method = "periodic"), col = "blue")
63
64 # Spline "fmm" (Forsythe, Malcolm, Moler)
65 plot(x3, y3, main = "Spline FMM", pch = 19)
66 lines(spline(x3, y3, method = "fmm"), col = "green")
67
68 # Spline "monoH.FC" (monotono)
69 plot(x3, y3, main = "Spline Mon tono", pch = 19)
70 lines(spline(x3, y3, method = "monoH.FC"), col = "purple")

```

7. Conclusión

7.1. Resumen Comparativo de Métodos

Método	Grado	Ventajas	Desventajas
Lineal	1	Simple, rápido	Poca precisión
Lagrange	n	Exacto en puntos	Inestable con muchos puntos
Newton	n	Fácil añadir puntos	Requiere cálculos previos
Cuadrática	2	Balance precisión/complejidad	Solo 3 puntos
Spline Cúbico	3	Muy suave, estable	Más complejo

Cuadro 1: Comparación de métodos de interpolación

7.2. Recomendaciones de Uso

1. **Interpolación Lineal:** Para estimaciones rápidas con pocos datos
2. **Lagrange/Newton:** Cuando se necesita exactitud en puntos específicos
3. **Interpolación Cuadrática:** Para patrones aproximadamente parabólicos
4. **Splines Cúbicos:** Para gráficos suaves y datos experimentales

7.3. Aplicaciones Prácticas

- **Ciencia de Datos:** Completar valores faltantes en series temporales
- **Ingeniería:** Diseño de curvas en CAD/CAM
- **Gráficos por Computadora:** Animaciones y modelado 3D
- **Finanzas:** Estimación de precios entre puntos de tiempo discretos

7.4. Perspectivas Futuras

Los métodos de interpolación continúan evolucionando con:

- Técnicas de interpolación multivariada
- Interpolación con restricciones físicas
- Métodos adaptativos basados en aprendizaje automático
- Interpolación en tiempo real para aplicaciones IoT

7.5. Código Final Integrado

```
1 # Funci n unificada de interpolaci n
2 interpol <- function(x_vals, y_vals, x_nuevo, metodo = "
   lineal") {
3   n <- length(x_vals)
4
5   if (metodo == "lineal") {
6     for (i in 1:(n-1)) {
7       if (x_nuevo >= x_vals[i] && x_nuevo <= x_vals[i+1])
8         {
9           return(y_vals[i] + ((y_vals[i+1] - y_vals[i]) /
10             (x_vals[i+1] - x_vals[i])) * (x_nuevo -
11               x_vals[i]))
12         }
13     }
14   }
15
16   if (metodo == "lagrange") {
17     resultado <- 0
18     for (i in 1:n) {
19       L_i <- 1
20       for (j in 1:n) {
21         if (j != i) {
22           L_i <- L_i * (x_nuevo - x_vals[j]) / (x_
23             vals[i] - x_vals[j])
24         }
25       }
26       resultado <- resultado + y_vals[i] * L_i
27     }
28     return(resultado)
29   }
30
31   if (metodo == "spline") {
32     spline_fit <- spline(x_vals, y_vals, n = 1000)
33     idx <- which.min(abs(spline_fit$x - x_nuevo))
34     return(spline_fit$y[idx])
35   }
36
37   return(NA)
38 }
39
40 # Ejemplo de uso
41 x <- c(1, 2, 3, 4, 5)
42 y <- c(2, 3, 5, 7, 11)
43
44 cat("Interpolaci n lineal en x=3.5:", interpol(x, y, 3.5, "
   lineal"), "\n")
45 cat("Interpolaci n Lagrange en x=3.5:", interpol(x, y, 3.5,
   "lagrange"), "\n")
```

```

43 cat("Interpolaci n Spline en x=3.5:", interpolar(x, y, 3.5, "
    spline"), "\n")
44
45 # Visualizaci n comparativa
46 x_seq <- seq(min(x), max(x), length.out = 100)
47 y_lineal <- sapply(x_seq, function(z) interpolar(x, y, z, "
    lineal"))
48 y_lagrange <- sapply(x_seq, function(z) interpolar(x, y, z, "
    lagrange"))
49 y_spline <- sapply(x_seq, function(z) interpolar(x, y, z, "
    spline"))
50
51 plot(x, y, pch = 19, col = "black", cex = 1.5,
52      main = "Comparaci n de M todos de Interpolaci n",
53      xlab = "x", ylab = "y")
54 lines(x_seq, y_lineal, col = "blue", lwd = 1, lty = 2)
55 lines(x_seq, y_lagrange, col = "red", lwd = 2)
56 lines(x_seq, y_spline, col = "green", lwd = 2)
57 legend("topleft", legend = c("Datos", "Lineal", "Lagrange", "
    Spline"),
58      col = c("black", "blue", "red", "green"),
59      pch = c(19, NA, NA, NA), lty = c(NA, 2, 1, 1))

```